

Question 2

How the system works, what was tested, and what has not been measured yet.

The question

How does it work, and what have you measured?

My answer

I should say the most important thing first. **I have not measured this in a real plant.** Everything I have comes from a model I built and tested, using published figures, values derived from those, and a handful of assumptions I have labelled as assumptions. The product is built to show that distinction on screen rather than bury it.

What the system does is this.

It takes a day's order book, meaning the dye lots with their shade depths, fabric weights and ship dates, plus the machine's water allowance. Then for each possible way of running that day it works out the consequences: water used, salt going into the effluent, how much concentrated reject that produces, evaporator heat, steam, carbon, cost, and whether anything ships late.

The link that does the real work is the reject calculation. Reject volume is not estimated. It comes out of salt mass divided by the highest concentration the membranes can safely reach, which I model at 60,000 mg/L. That number is a model parameter, not a claim that every membrane in Tirupur sits at exactly that limit. Reject volume then determines evaporator heat, which determines steam, fuel and carbon.

For the five-lot example the system evaluates 480 candidate plans, every running order against every process strategy, and picks the best one that is actually feasible. Not the one with the smallest water number. Ship dates are hard constraints, so a plan cannot buy an environmental saving by becoming something the factory cannot actually run.

There are two human gates. A named person approves the plan. A second gate controls wash-off release, and if a sensor is behaving oddly the system holds instead of releasing, and credits zero saving.

I tested it three ways.

Automated tests. 180 of them, covering the calculations, the accounting invariants, the safety interlocks, record integrity, fault behaviour and concurrency.

A check against published plant data. The Central Pollution Control Board measured 18,340 mg/L of dissolved solids entering evaporation at a Tirupur unit. Separately, operators report reject running at 20 to 30 percent of inlet volume. Those two facts come from different places and neither is an input to my engine. When I feed in the first, the model predicts 30.6 percent, which is inside the reported band. At 12,000 mg/L it gives exactly 20.0 percent, and at 15,000 it gives 25.0. Nothing was adjusted to make that line up.

It is a sanity check against published operating data. It is not field validation, and I would not describe it as one.

A worked example. One machine, one shift, five lots. The baseline plan draws 11,907 litres of fresh water. The recommended plan draws 11,187 and brings evaporator heat down from 2,239 to 2,104 kWh. There is a plan available that uses only 5,889 litres, and the system refuses it, because it makes one delivery 2.6 hours late.

Of the 25 inputs driving the model, 17 come from published sources, 4 are derived from those, and 4 are openly flagged assumptions. None of them are plant measurements. The two measurements I want first are evaporator steam flow and reject conductivity.

Supporting notes

Why the answer is shaped this way

The question asks what was measured, and the truthful answer is nothing, in a plant. Blurring that would collapse the moment a judge asked one direct follow-up, so it goes in the opening line. Saying it voluntarily buys credibility for everything after it.

The three-part test structure exists because "I tested it" means very little on its own. The automated tests show internal consistency. The CPCB comparison shows the physics behaves like real plants. The worked example shows it produces a decision rather than a report. Those answer three different doubts, and a judge may only hold one of them.

The refused plan is included on purpose. A system turning down its own best water number is the clearest evidence available that it was built for a factory rather than for a scoreboard.

Every figure, and how to check it

Claim	Value	Where to verify
Automated tests	180	<code>python -m pytest tests/ -q</code>
Candidate plans evaluated	480	<code>figures.json</code> → candidates; 120 orders × 4 strategies
Membrane concentration ceiling	60,000 mg/L	<code>core/factors.py</code> → <code>ro_max_reject_tds_mg_l</code> , inside a published 15,000–80,000 band
CPCB measured inlet	18,340 mg/L	CPCB Tirupur ZLD assessment
Model prediction at that inlet	30.6%	<code>tests/test_published_validation.py</code>
At 12,000 and 15,000 mg/L	20.0% and 25.0%	same test
Published reject band	20–30% of inlet	Indian ZLD operator data
Baseline freshwater	11,906.8 L	<code>figures.json</code> → <code>options.OPTION_A.fresh</code>
Recommended plan freshwater	11,186.8 L	<code>options.OPTION_B.fresh</code>

Claim	Value	Where to verify
Evaporator heat, baseline to recommended	2,239.4 → 2,104.0 kWh	<code>options.OPTION_A/B.mee</code>
Lowest-water plan	5,888.8 L, 2.6 h late, refused	<code>options.OPTION_C</code>
Evidence mix	17 published, 4 derived, 4 assumed, 0 measured	<code>core/factors.py</code> , <code>/api/evidence</code>

What the answer deliberately avoids claiming

- The CPCB comparison is not described as validating the product. It validates the physics, at conditions somebody else measured, and says nothing about any particular site.
- 60,000 mg/L is not presented as a universal figure. It is a modelled ceiling sitting inside a published range.
- The automated tests are not claimed to prove real-world correctness. They prove the system is internally consistent and fails safe.

The hardest follow-up

"Your model agrees with your model. Why would I believe any of it?"

Because the check runs on numbers the engine never reads. CPCB measured the inlet. Operators reported the reject band. I feed in the first and compare against the second, with nothing in between tuned to make them meet. If the salt-mass formulation were wrong that test would fail, and because it is a test rather than a slide, it fails the build.